得分	
评分人	

5 Don't Break The Fourth Wall (19 pts)

5.1 Introduction (4 pts)

In this problem, we define an abstract data type (ADT) called `lst` to simulate the behavior of Python lists. Read the provided code carefully to understand how the `lst` ADT is implemented using higher-order functions. The insight of the implementation is that a list is an ordered collection of items that can be viewed as a function from indices to items. For example, the list `[1, 4, 5]` can be viewed as a function f with $f(0) = 1$, $f(1) = 4$, and $f(2) = 5$.

We define the four functions below (`lst_empty`, `lst_add`, `lst_len`, and `lst_get`) as the interface of the `lst` ADT. That is, though the evil TAs may later replace the implementation of these functions using other things rather than lambdas, the behavior of these functions will remain as specified in their docstrings.

First, check your understanding of the code by implementing the `lst_get` function in the code snippet below.

```
1  def lst_empty():
2      """Return an empty lst."""
3      return lambda i : None
4
5  def lst_add(l, item):
6      """Return a new lst with item added to the front of the lst l."""
7      return lambda i : item if i == 0 else l(i - 1)
8
9  def lst_get(l, index):
10     """Return the item at position index in the lst l."""
11     return None if index < 0 else l(index) （1分）
12
13 def lst_len(l):
14     """Return the length of the lst l."""
15     index = 0
16     while lst_get(l, index) is not None:
17         index += 1
18     return index
```

Then, check your understanding of the code by answering the following WWPD (What Would Python Display?) questions. For each expression, write down the displayed value when the expression is evaluated in a Python interpreter.

Expression	Interactive Output
`lst_len(lst_empty())`	`0`
`lst_len(lst_add(lst_add(lst_empty(), 1), 2))`	2 （1分）
`lst_len(lst_add(lst_add(lst_empty(), None), 2))`	1 （1分）
`lst_len(lst_add(lst_add(lst_empty(), 1), None))`	0 （1分）

Note that for simplicity, you do not need to consider nested lists in this problem, that is, lists containing other lists as items (e.g., `[1, [2, 3], 4]`).

5.2 Find the Wall Breakers (10 pts)

ADT separates its interface from its implementation. It is essential not to break the abstraction barrier of an ADT, that is, not to rely on the internal implementation details of the ADT rather than its interface.

The following examples implement some common list operations using the `lst` ADT. Your task is to identify the good implementations that (1) correctly implement the specified functionality and (2) do not break the abstraction barrier of the `lst` ADT. For each implementation, write an "T" if the implementation is good; otherwise, write an "F".

Implementation	Is Good?

`def lst_contains(l, item):`
`  """Return True if item is in the lst l, False otherwise."""`
`  for i in range(lst_len(l)):`
`  if l(i) == item:`
`  return True`
`  return False`
	F （2分）

`def lst_reverse(l):`
`  """Return a new lst that is the reverse of the lst l."""`
`  result = lst_empty()`
`  for i in range(lst_len(l)):`
`  result = lst_add(result, lst_get(l, i))`
`  return result`
	T （2分）

`def lst_append(l, item):`
`  """Return a new lst with item added to the end of l."""`
`  return lambda i : item if i == lst_len(l) else lst_get(l, i)`
	F （2分）

`def lst_to_list(l):`
`  """`
`  Convert all lst abstraction to Python lists, and return a Python list`
`  containing all items in the lst l with the order of items preserved.`
`  """`
`  return [lst_get(l, i) for i in range(lst_len(l))]`
	T （2分）

`def lst_filter(l, f):`
`  """`
`  Return a new lst that only keeps the items in the lst l that`
`  make the function f(item) returns True, and preserve their order in l.`
`  """`
`  result = lst_empty()`
`  for i in range(lst_len(l)):`
`  if f(lst_get(l, i)):`
`  result = lst_add(result, lst_get(l, i))`
`  return result`
	F （2分）

5.3 I Can Do Better than the Noobs (5 pts)

In the previous problem, you have identified many bad implementations written by the TAs. Rather than reviewing their bad code, you may already come up with the idea that "I can do better than those noobs!".

Now it's your turn to solve some problems using the `lst` ADT. Note that the code should (1) correctly implement the specified functionality and (2) not break the abstraction barrier of the `lst` ADT. You need to fill in the blanks in the following code snippets such that the code meets the two requirements above.

First, implement another version of the `lst_append` function in the previous problem:

```
def lst_append(l, item):
    """Return a new lst with item added to the end of l."""
    result = lst_add(lst_empty(), item) （1分）
    for i in reversed(range(lst_len(l))) （1分）:
        result = lst_add(result, lst_get(l, i))
    return result
```

Then, use the `lst_append` function you just implemented to solve a more complex problem:

```
def lst_interleave(l1, l2):
    """
    Given two lsts l1 and l2, return a new lst that interleaves items from l1 and l2.
    If one lst is longer than the other, append the remaining items at the end.
    For example,
        lst_interleave(
            lst_add(lst_add(lst_empty(), 1), 1),
            lst_add(lst_add(lst_add(lst_empty(), 2), 2), 2))
    would return a lst equivalent to
        lst_add(lst_add(lst_add(lst_add(lst_add(lst_empty(), 2), 2), 1), 2), 1)
    """
    result = lst_empty()
    i, j = 0, 0
    while True:
        if lst_get(l1, i) is None and lst_get(l2, j) is None （1分）:
            return result
        if lst_get(l1, i) is not None （1分）:
            result = lst_append(result, lst_get(l1, i))
            i += 1
        if lst_get(l2, j) is not None （1分）:
            result = lst_append(result, lst_get(l2, j))
            j += 1
```